

Becoming a dbt Architect



Will Weld | Senior Resident Architect



Dr Akash Trivedi | Lead Technical Instructor

September 15-18, 2026

Learning outcomes

By the end of this training, you'll be able to:

- Explain how dbt integrates with various data platforms and git providers
- Leverage security & compliance features based on your team's needs
- Deploy and orchestrate dbt environments and jobs
- Use dbt APIs and webhooks to integrate, automate, and extend dbt workflows
- Understand and implement components of a data governance framework
- Bonus: Familiarize yourself with the format and structure of the dbt Architect exam

What is a dbt architect?

This training will serve as an introductory course for dbt Architects to learn foundational concepts covered in the dbt Architect certification exam.

Key responsibilities of a dbt Architect:

- Designing secure, scalable implementations
- Integrating dbt with enterprise platforms and services
- Orchestrating environments (dev, staging, prod)
- Configuring role-based access control (RBAC)
- Managing integrations and collaborative development workflows

How today works

...and what we won't cover

Format

- Slides, discussion, demos, labs
- Open to questions throughout

Out of scope

- e.g: Jobs-as-code, airflow, terraform, deep admin, etc.

Get your laptops ready!

Agenda

01

Introduction to the dbt platform

02

Setting up dbt connections

03

Configuring and managing dbt projects

04

Security & monitoring

05

Governance and discovery

06

Core exam concepts

Prerequisites

Required

- dbt Fundamentals
- Git Fundamentals
- ADLC awareness
- SQL proficiency

Nice to have

- dbt AE certification
- dbt Mesh course
- dbt Catalog course
- Advanced Deployment course



01

Introduction to the dbt platform

- Why architecture matters at scale
- Data platform functions
- A managed experience

Why architecture matters at scale

Every architectural choice carries a real downstream cost – in engineering hours, reliability, and how fast your team ships

Self-managing the whole stack
(infrastructure, orchestration, platform config, CI/CD) is a large, ongoing effort

The architect's job: make those choices deliberately, with the tradeoffs in view

Data platform functions

why an architect cares

01

Cloud infrastructure

the backbone: compute, networking, storage, regions

→ *cares about: multi-region, private connections*

02

Platform configuration

environments, resources, access controls

→ *cares about: governance, service accounts, logical separation*

03

Orchestration

scheduling, chaining, monitoring, integrations

→ *cares about: robust, event-driven workflow design, troubleshooting*

04

CI/CD

automated build, test, deploy of code & config

→ *cares about: pipeline setup, tool integration, troubleshooting*

The dbt platform: a managed experience

A managed, scalable, first class experience of dbt across the lifecycle



One managed platform for the four functions – so your team doesn't stand them up from scratch



02

Setting up dbt connections

- Working with data platforms
- Working with git
- Security & network surface preview
-  Hands-on lab

Working with data platforms

Data Platform Adapters

Data Platform Adapters allow you to connect your dbt project to your platform of choice

- Trusted Adapters
- Community Adapters

Trusted adapters



AlloyDB

- [Set up in the dbt platform](#)
- [Install with dbt Core](#)

pypi package 1.11.0

dbt platform dbt Core



Apache Spark

- [Set up in the dbt platform](#)
- [Install with dbt Core](#)
- [Install with dbt Fusion](#)

pypi package 1.11.0

dbt platform dbt Core
Fusion



Athena

- [Set up in the dbt platform](#)
- [Install with dbt Core](#)

pypi package 1.11.0

dbt platform dbt Core

Azure Synapse

- [Set up in the dbt platform](#)
- [Install with dbt Core](#)

pypi package 1.8.5

dbt platform dbt Core



BigQuery

- [Set up in the dbt platform](#)
- [Install with dbt Core](#)
- [Install with dbt Fusion](#)

pypi package 1.12.0

dbt platform dbt Core
Fusion



ClickHouse

- [Install with dbt Core](#)

pypi package 1.10.2

dbt Core



Databricks

- [Set up in the dbt platform](#)
- [Install with dbt Core](#)
- [Install with dbt Fusion](#)

pypi package 1.12.4

dbt platform dbt Core
Fusion



DuckDB

- [Install with dbt Core](#)
- [Install with dbt Fusion](#)

pypi package 1.11.0

dbt Core Fusion



Dremio

- [Install with dbt Core](#)

pypi package 1.10.1

dbt Core



Glue

- [Install with dbt Core](#)

pypi package 1.10.19

dbt Core



Exasol

- [Install with dbt Core](#)

pypi package 1.12.0

dbt Core

IBM Db2

- [Install with dbt Core](#)

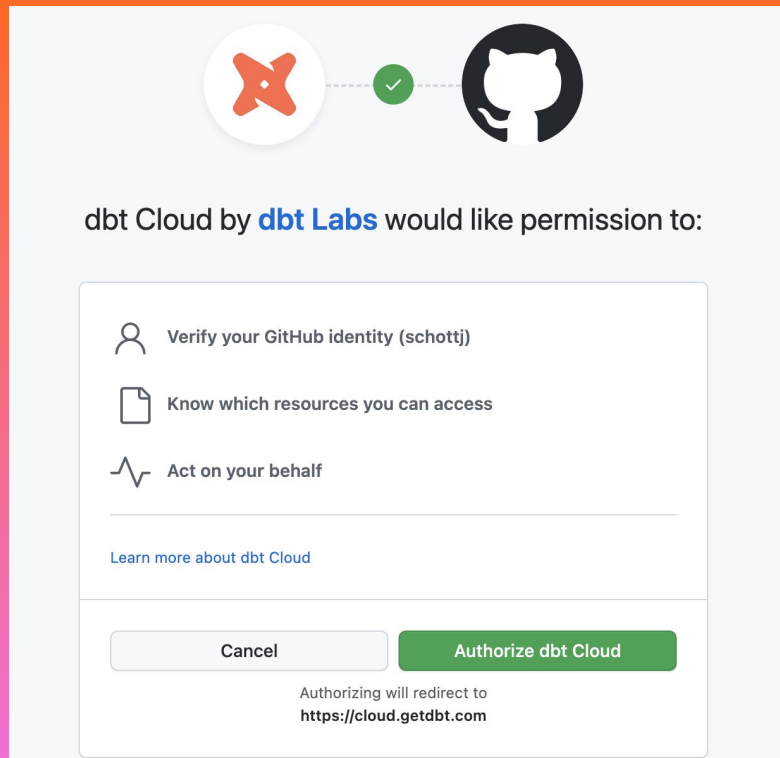
pypi package 1.0.0

dbt Core

Working with git

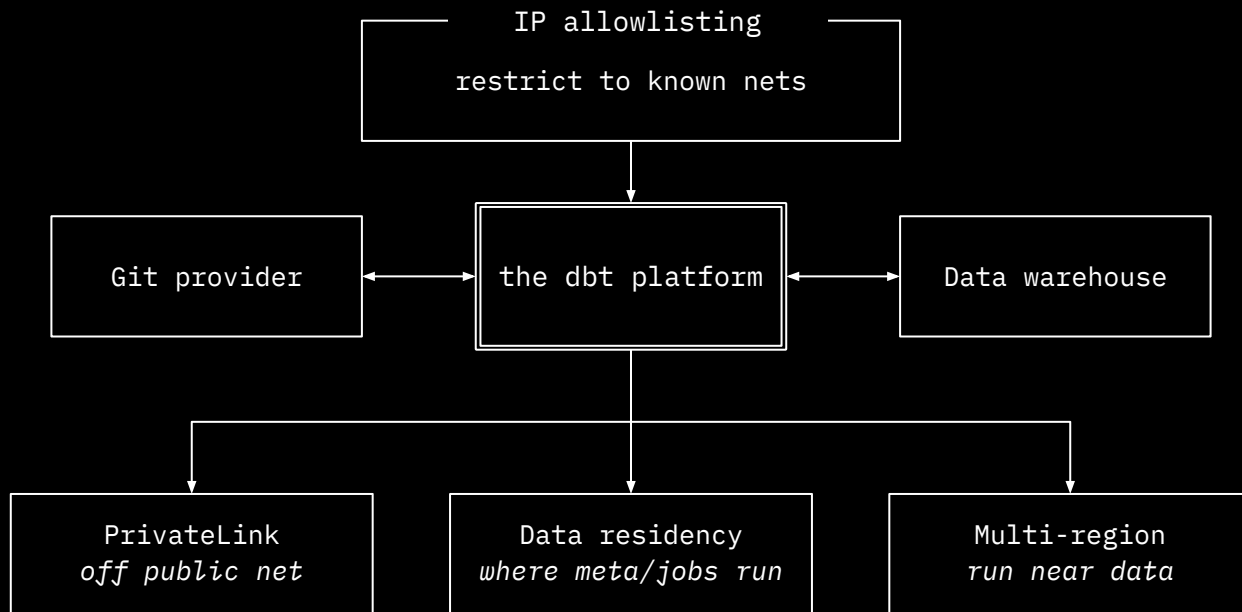
Version control is handled through git integrations

- Managed Repo
- Git Clone
- GitHub
- GitLab
- Azure DevOps



The security & network surface

Connecting dbt opens a security surface worth naming – we'll go deeper in Section 3

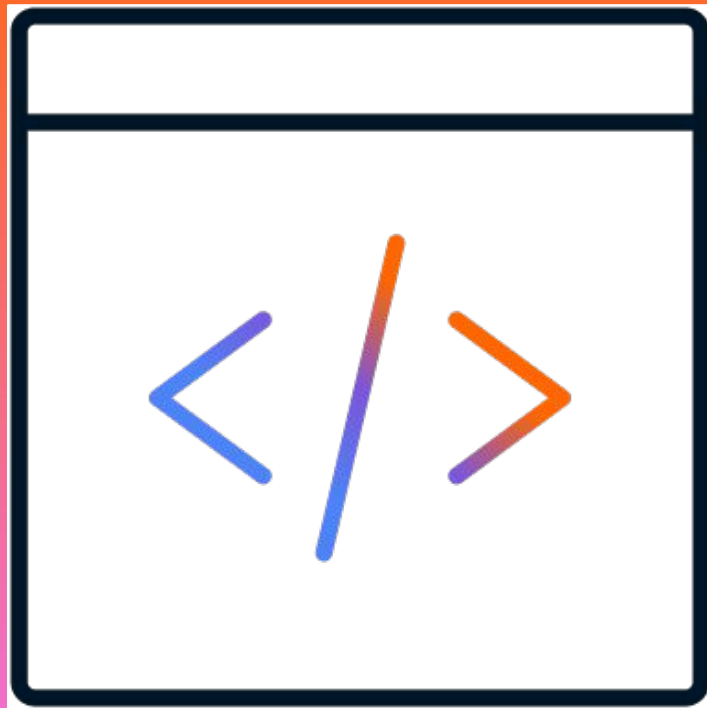


Hands-On #1

Account Setup

This is how you get started:

1. Sign out of any dbt platform account you already have or use an incognito/private browser
2. Go to <http://workshops.us1.dbt.com/workshop>
 - a. Enter your name & Select the **Becoming a dbt Architect** training from the drop down menu
 - b. Enter the course passcode: **Summit2026!**
 - c. Select **Complete Registration**
3. Make a note of the dbt platform account Email Address and Password. You will need this to log back into your account in the future.
4. Log in to access your dbt platform account
5. Create a git branch **s26_arch_firstlast**



Sample Question

Congratulations! Your company has expanded to Europe. As part of this expansion, IT plans to set up another data warehouse in the EU while leveraging the same Enterprise dbt account across regions.

What should you do to achieve this?

- A. Purchase another account for the EU region.
- B. Configure a connection to the EU – no further action is required.
- C. Upgrade your account to Business Critical and establish a PrivateLink connection.
- D. Configure dbt Mesh Cross Platform to connect to the EU warehouse.

Sample Question

Congratulations! Your company has expanded to Europe. As part of this expansion, IT plans to set up another data warehouse in the EU while leveraging the same Enterprise dbt account across regions.

What should you do to achieve this?

- A. Purchase another account for the EU region.
- B. Configure a connection to the EU – no further action is required.
- C. Upgrade your account to Business Critical and establish a PrivateLink connection.
- D. Configure dbt Mesh Cross Platform to connect to the EU warehouse.

Sample Question

Explanation

In a multi-Region Setup, it would be prudent to set up two separate PrivateLink endpoints in the regions in which you are established. You will then configure multiple projects to use the respective private endpoints. This helps you maintain data sovereignty (EU data stays in EU) and isolation (no data traverses public networks).



03

Configuring & managing projects

- How to work with environments in dbt
- Environment variables
- Git strategies
- Deployment strategies
- Jobs & orchestration
- [DEMO] Project variables
- [HOL] Environments & env vars, job design

dbt Environments

A dbt Environment is an isolated space where you can run your code using specific configurations.

- In the dbt platform, there are two types of environments:
- **Development**
 - Only one per project, because it directly informs the IDE
 - Uses the user's credentials to connect to the data platform
 - Builds objects in the connection's database and user's schema
- Deployment

Environment - Development



dbt Version



git branch



connection

Project Settings



Connections

User Settings



Credentials



Who do I login as?: user
Where do I build to?: database.schema

dbt Environments

A dbt Environment is an isolated space where you can run your code using specific configurations.

- In the dbt platform, there are two types of environments:
- Development
- **Deployment**
 - Main purpose is for automation
 - Same settings that a development env has, and:
 - Accepts additional information such as connection overrides, schema, and user (service acct)

Environment - Deployment



dbt Version



git branch



connection

- * connection overrides
- * default schema
- * who to run as (svc)

dbt Deployment environment types

01

Staging

The purpose of a staging environment is for final integration testing using production source data

Typically maps to a git branch like staging, qa, or uat

02

Production

This is where your production code runs and serves live business data

Maps to the default git branch, typically called main

03

General

The General deployment environment type is typically used for flexible, multi-purpose deployment scenarios that don't fit the standard staging or production patterns

Environment variables

Overview

- Parameterize dbt behavior so the same code runs differently per environment
- Set at four levels – project · environment · job · personal – resolved by an order of precedence
- Naming: `dbt_` is visible to anyone · `dbt_env_secret_` is hidden and secured
- Use `{{ env_var('DBT_MY_VAR') }}` in any place dbt processes Jinja code, such as `dbt_project.yml`, `sources.yml`, `schema.yml`, `model.sql`



Personal Override	Job Override
Development Environment	Deployment Environments
Project Default	
Jinja Function Default Arg	

Environment variables

Common usages at a glance



Credentials

store secrets with `DBT_ENV_SECRET_`
rotate without touching code



Materialize

dev env → view
prod env → table



Resource limits

row caps on dev/CI jobs
full data in prod runs



Dynamic warehousing

dev vs prod warehouse sizing



Schema / database switching

build models in the right place per environment

Special environment variables

Run details

<code>DBT_CLOUD_PROJECT_ID</code>	The ID of the dbt platform Project for this run
<code>DBT_CLOUD_JOB_ID</code>	The ID of the dbt platform Job for this run
<code>DBT_CLOUD_RUN_ID</code>	The ID of this particular run
<code>DBT_CLOUD_RUN_REASON_CATEGORY</code>	The trigger category (eg. scheduled, github_pull_request)
<code>DBT_CLOUD_RUN_REASON</code>	The specific trigger for this run (eg. scheduled, custom via API)
<code>DBT_CLOUD_ENVIRONMENT_ID</code>	The ID of the environment for this run
<code>DBT_CLOUD_ACCOUNT_ID</code>	The ID of the dbt platform account for this run

Git details

<code>DBT_CLOUD_PR_ID</code>	The Pull Request ID in the connected version control system
<code>DBT_CLOUD_GIT_SHA</code>	The git commit SHA which is being run for this Pull Request build

* Git details only available for GitHub, GitLab, and Azure DevOps PR builds triggered via a webhook

Hands-On #2

Environment Variables

Build one:

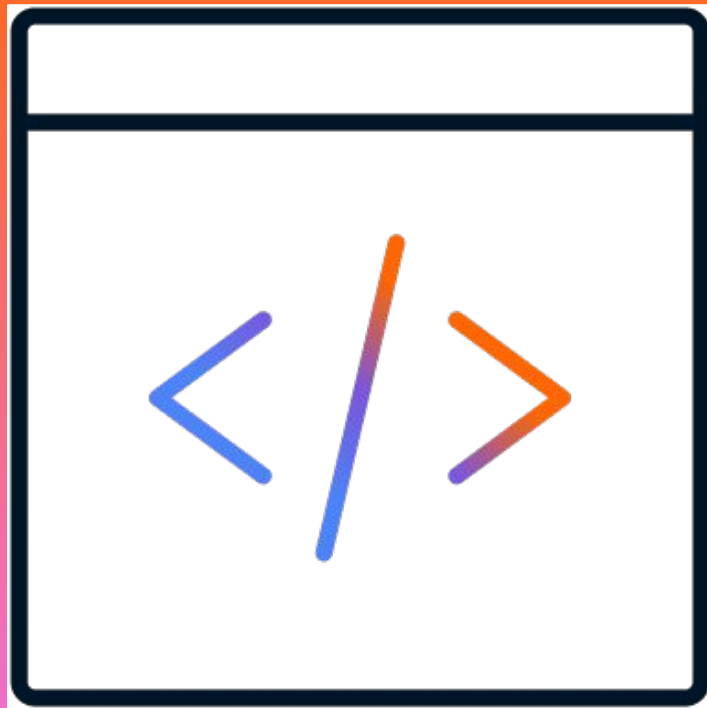
1. In the Foundational Project, navigate to `models`→`DEMO_ENV_VAR`→`my_first_dbt_model.sql`
2. `dbt build` it... Check logs? How did it materialize? Why?

Define environment variable:

1. Navigate to `Orchestration`→`Environments`
2. Add a `DBT_MATERIALIZATION` environment variable that materializes the model in the dev environment as a table and save it

Build two:

1. Add an `env_var()` to change how `my_first_dbt_model` materializes
2. `dbt build` again... What changed in the logs?



Demo #1

Project Variables

1. Review the [Project Variables](#) documentation
2. Navigate to `dbt_project.yml` and add a `min_id` project variable with a default of 1
3. Navigate to `models→DEMO_ENV_VAR→my_second_dbt_model.sql` to change the filter to accept ids greater than or equal to the `min_id` variable
4. `dbt build` this and its upstream dependencies. What do you see in `Preview`?
5. Run a build command that uses a different value for the `min_id`
6. Observe the results. Do you see the change?
7. Comment out the `vars` config in `dbt_project.yml` and instead add to `vars.yml`
8. Redo step 5 and observe the outcome
9. What happens when you have `vars` configs in both the top level `vars.yml` and `dbt_project.yml` files?



Sample Question

Inspect this environment variable configuration:

Key	Project default	Development	UAT	Production
DBT_ENV_VALUE		dev		prod

Within a job there is an override to this variable with the value `finance`. Which value will be used when running the dbt job if this job configuration is removed?

- A. The dbt project default `not_set`.
- B. The default argument supplied to the `env_var` Jinja function in code.
- C. The value assigned to the environment specified in the job description.
- D. The job-level environment variable is required and cannot be removed.

Sample Question

Inspect this environment variable configuration:

Key	Project default	Development	UAT	Production
DBT_ENV_VALUE		dev		prod

Within a job there is an override to this variable with the value finance. Which value will be used when running the dbt job if this job configuration is removed?

- A. The dbt project default not_set.
- B. The default argument supplied to the env_var Jinja function in code.
- C. The value assigned to the environment specified in the job description.
- D. The job-level environment variable is required and cannot be removed.

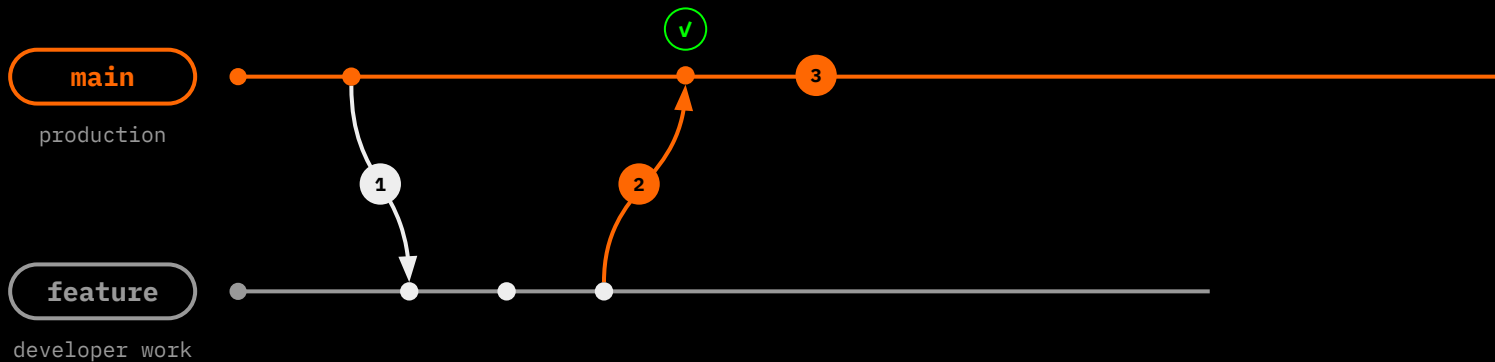
Sample Question

Explanation

The Project Default column is recommended when you want to supply a catch-all default or add a project-wide token or secret. Values set at the environmental level take priority over the project-level default value, and as seen in the above example, dbt will interpret the variable value differently in your Production vs. Development environment.

A job inherits the values set at the environment level, but a job-level override will take precedence for the environment in which it runs. If you remove the job-level override, it will default back to the environment variable, in this case, the value prod.

Direct Promotion



✓ PR review is the only gate - tested code ships straight to production

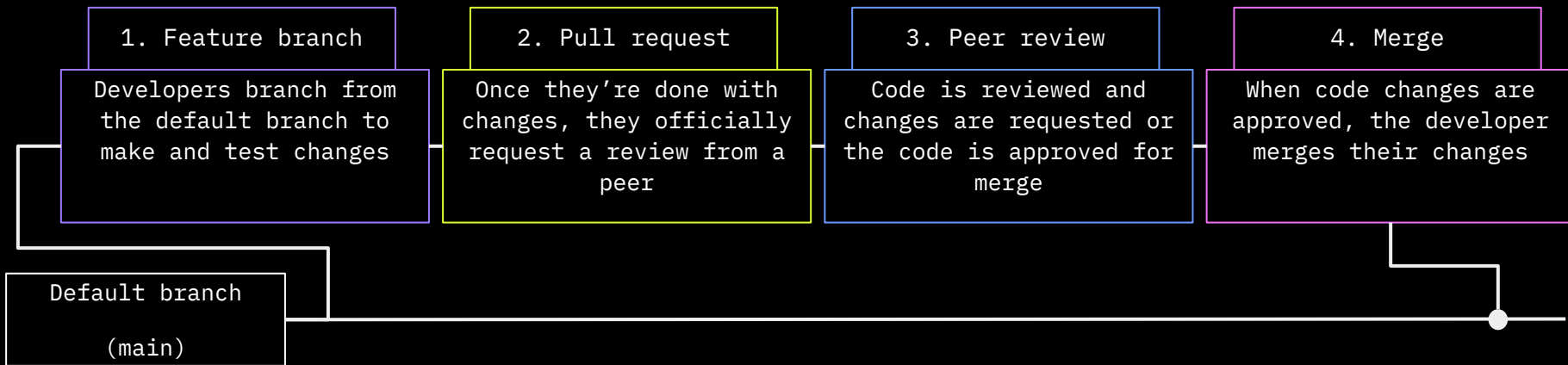
1. branch feature off **main**

2. PR → CI + peer review, merge to **main**

3. deploy production

Direct Promotion (Continuous Delivery)

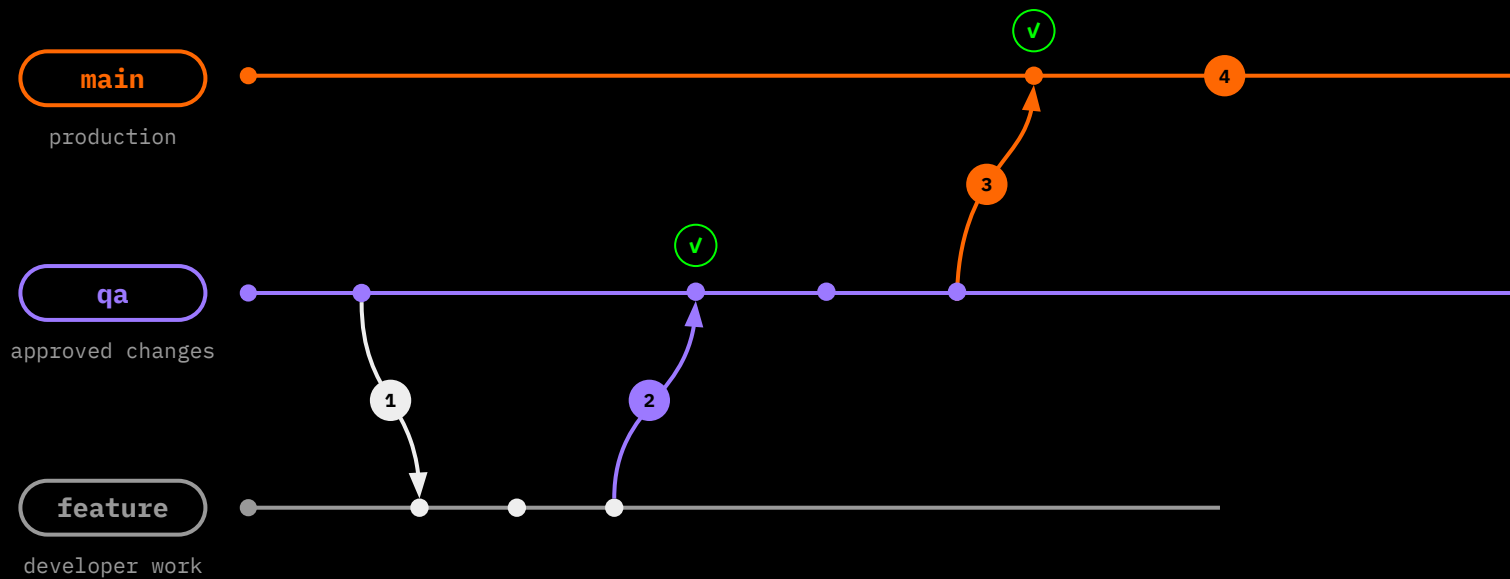
Direct Promotion means we only keep one long-lived branch in our repository – in this case, main



Branch layout

- Feature - the developer's unique branch for tasks
- Main - the production version of code

Indirect Promotion

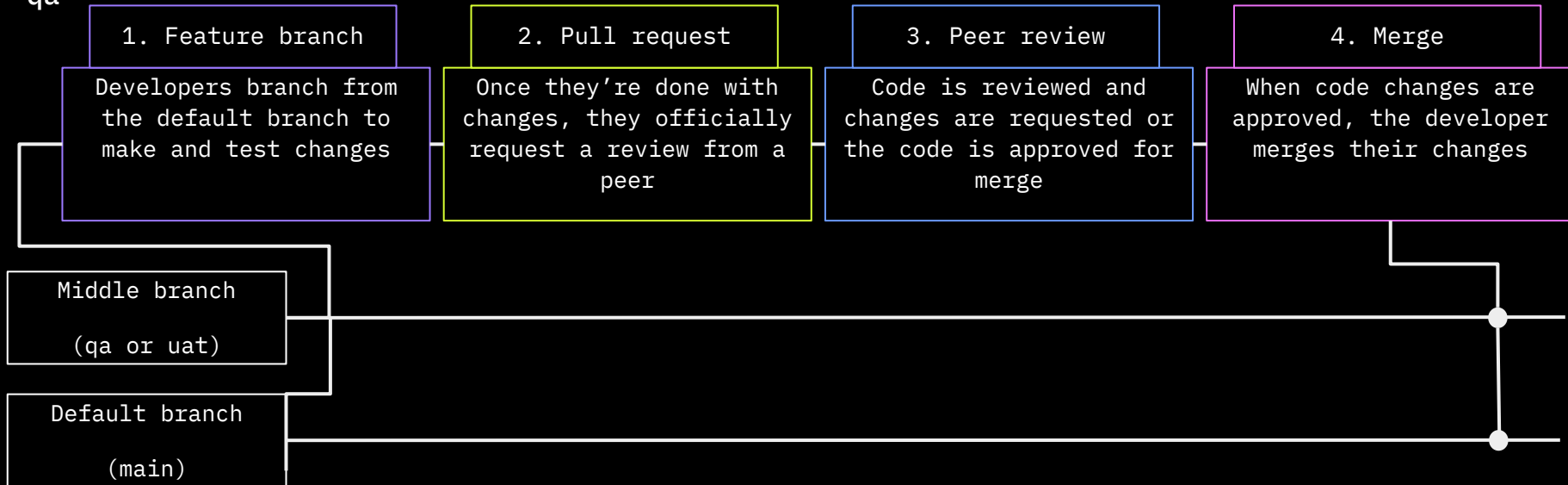


(v) required review gate before merge is allowed

1. branch feature off qa
2. PR → CI + peer review, merge to qa
3. release PR: qa → main
4. deploy prod

Indirect Promotion (Continuous Integration)

Indirect Promotion adds other long-lived branches that derive from main – in this case, qa



Branch layout

- Feature – the developer's unique branch for tasks
- qa – approved changes from feature branches
- Main – the production version of code

Branching strategy comparison

Direct Promotion

Strengths

- Changes get pushed to production efficiently
- Code ownership at the developer level
- What's tested is what goes into production

Weaknesses

- Not a best practice from a QA perspective
- Missing layer for peers & SMEs to review
- No environment to test changes before production

Indirect Promotion

Strengths

- Better approach to test end-to-end changes
- Centralized and shared space to simulate production
- Tighter control over changes

Weaknesses

- Slower to move changes to production
- Processes/ownership have to be put in place (especially when it gets out of sync)
- Additional environment = more compute resources

Setting branches for environments

To tie together our deployment environments with specific Git branches, we can set a custom branch option for each of our environments.

Environment name

STAGING_C25_ARCHITECT

Environment type

Deployment

This project already has a development environment, only deployment environments can be created.

Set deployment type ?

Designates the deployment environment type.

General **STG Staging** PROD Production

dbt version ?

Latest

dbt automatically upgrades the dbt version running in this environment, based on the release track you select.

☒ Only run on a custom branch

Custom branch

stage_c25_architect

Sample Question

Your team runs a Direct Promotion workflow with `main` as the only long-lived branch. Compliance now requires that every change be validated against production source data in a shared environment before it reaches production.

Which change best meets this requirement?

- A. Add a staging deployment environment pointed at `main` and schedule a nightly job against it.
- B. Add a long-lived `qa` branch, create a staging deployment environment with "Only run on a custom branch" set to `qa`, and require a release PR from `qa` to `main`.
- C. Enable "Run compare changes" on the existing CI job so reviewers can see row-level differences before merge.
- D. Give each developer a personal deployment environment that builds their feature branch into a shared schema.

Sample Question

Your team runs a Direct Promotion workflow with `main` as the only long-lived branch. Compliance now requires that every change be validated against production source data in a shared environment before it reaches production.

Which change best meets this requirement?

- A. Add a staging deployment environment pointed at `main` and schedule a nightly job against it.
- B. Add a long-lived `qa` branch, create a staging deployment environment with "Only run on a custom branch" set to `qa`, and require a release PR from `qa` to `main`.
- C. Enable "Run compare changes" on the existing CI job so reviewers can see row-level differences before merge.
- D. Give each developer a personal deployment environment that builds their feature branch into a shared schema.

Sample Question

Explanation

This is Indirect Promotion. The gate has to be both a long-lived branch and a deployment environment bound to that branch via the custom branch setting – that pairing is what makes the staging environment test something different from production. A staging environment still pointed at `main` (1) validates code that has already merged. Compare changes (3) is a review aid inside CI, not an integration environment. Per-developer deployment environments (4) reintroduce the isolation problem the release gate is meant to solve.

dbt Jobs

A dbt Job is a sequence of dbt commands that you define and then trigger in various ways.

A deployment environment can have 0:M jobs.

Job Triggers

- Scheduled
- After another job completes
- API call
- Git Integration: on PR or merge events
- Manual (button push)

Environment - Deployment

Jobs

Daily Run at 8:00am
`dbt build`

Sunday full refresh
`dbt build --full-refresh`

Jobs in deployment environments

To start setting up jobs, we need to know the answers to these four questions:

Job Property	Question
Command	What models do we want to build
Environment	Where do we get the code from to build it?
	Where do we want the objects to go?
Trigger	When do we want to build it?

dbt Job types

01 Deploy jobs

Purpose: build production data assets.

Triggers: schedule, API, or the successful completion of another job.

02 CI jobs

Purpose: build and test new code before merging changes into production.

Triggers: a commit to a PR or by API.

03 Merge jobs

Purpose: build merged changes into deployment branches (qa, prod) or update state for deferral.

Triggers: successful merge into the environment's branch or by API.

dbt Jobs comparison

	Deploy Jobs	CI Jobs	Merge Jobs
Purpose	Builds production data assets.	Builds and tests new code before merging changes into production.	Build merged changes into production or update state for deferral.
Trigger Types	Triggered by a schedule, API, or the successful completion of another job.	Triggered by a commit to a PR or by API.	Triggered by a successful merge into the environment's branch or by API.
Destination	Builds into a qa or production database and schema.	Builds into a staging database and ephemeral schema, lived for the lifetime of the PR.	Builds into a production database and schema.
Execution Mode	Runs execute sequentially, so as to not have collisions on the underlying DAG.	Runs execute in parallel to promote team velocity.	Runs execute sequentially, so as to not have collisions on the underlying DAG.
Efficiency Run Savings	Detects over-scheduled jobs and cancels unnecessary runs to avoid queue clog.	Cancels existing runs when a newer commit is pushed to avoid redundant work.	N/A
State Comparison	Only sometimes needs to detect state.	Almost always needs to compare state against the production environment to build on modified code and its dependents.	Almost always needs to compare state against the production environment to build on modified code and its dependents.

dbt Jobs commands > Deploy Jobs

Build production assets. This environment is the source of truth, so no defer.

```
# Standard scheduled production build (models + tests + snapshots + seeds)
dbt build
```

```
# Check source freshness first, then build (as two steps in the job)
dbt source freshness
dbt build
```

```
# Split cadences with tags across separate deploy jobs
dbt build --select tag:hourly
dbt build --select tag:daily
```

```
# Rebuild a single domain sub-DAG and its upstreams
dbt build --select +marts.finance
```

```
# Occasional full-refresh of incremental models (separate, off-hours job)
dbt build --select config.materialized:incremental --full-refresh
```

dbt Jobs commands > CI Jobs

Validate a PR before merge. Only build what changed + downstream, deferring the rest to prod/staging.

```
# In the dbt platform (defer + state auto-provided by the comparison env)
dbt build --select state:modified+

# Explicit dbt Core equivalent (shows the mechanics)
dbt build --select state:modified+ --defer --state path/to/prod-artifacts
```

dbt Jobs commands > Deploy Jobs

Build merged changes into the deployment branch (qa/main) and refresh artifacts so the next CI run has a current defer target.

```
# Build just the newly merged changes into the branch's environment
dbt build --select state:modified+

# Explicit dbt Core equivalent
dbt build --select state:modified+ --defer --state path/to/prev-branch-artifacts

# If the job's ONLY purpose is to refresh the deferral manifest (no build)
dbt parse          # or: dbt compile
```

dbt Runs

Runs, run history, run details

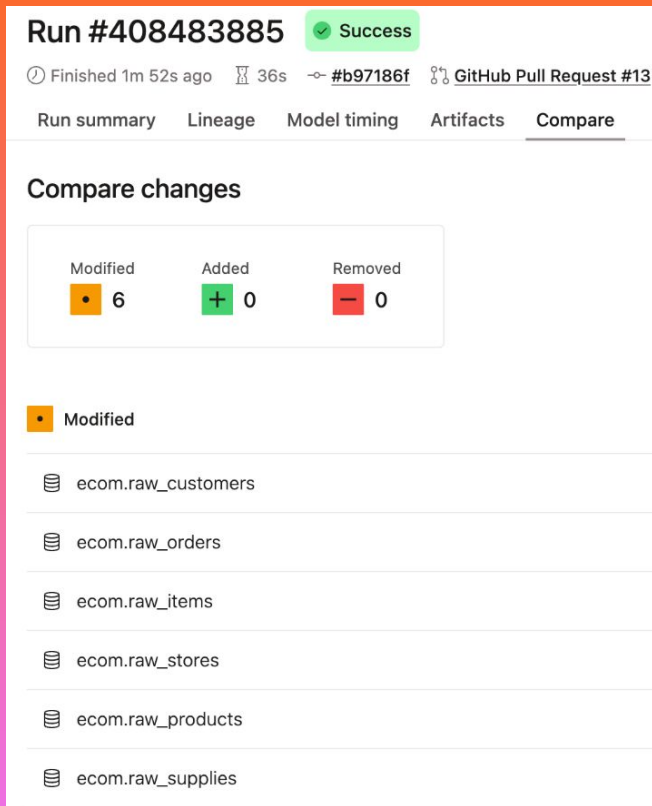
- A run = one execution of a job;
a job has 0-to-many runs, each
with a status: success · failure
· warning · cancelled
- Notifications can fire on any
status
- Find them all: Deploy → Run
History
- Click into a run for four tabs:
Summary · Lineage · Model timing
· Artifacts



Run details – Compare

The **Compare** tab is shown for **CI Job Runs** when the “Run compare changes” setting is enabled to show changes between what’s in production vs. the PR.

- **Overview** – High-level summary about the changes to the models such as the number of primary keys that were added or removed.
- **Primary keys** – Details about the changes to the records.
- **Modified rows** – Details about the modified rows. Click Show full preview to display all columns.
- **Columns** – Details about the changes to the columns.



The screenshot shows the 'Compare' tab for a CI job run. At the top, it displays 'Run #408483885' with a green 'Success' status. Below this, it shows 'Finished 1m 52s ago' and '36s' duration. A link to the pull request is provided: '#b97186f' linked to 'GitHub Pull Request #13'. Navigation tabs include 'Run summary', 'Lineage', 'Model timing', 'Artifacts', and 'Compare' (which is active). The 'Compare changes' section shows a summary: 6 Modified (orange square with a dot), 0 Added (green square with a plus), and 0 Removed (red square with a minus). Below this, a list of modified models is shown under the heading 'Modified' (orange square with a dot): ecom.raw_customers, ecom.raw_orders, ecom.raw_items, ecom.raw_stores, ecom.raw_products, and ecom.raw_supplies. Each model name is preceded by a document icon.

Sample Question

Your CI job runs `dbt build` on every pull request and now takes as long as the nightly production build. The job's environment has no deferral configured.

What should you change?

- A. Change the command to `dbt build --select state:modified+` and set the job's comparison environment to Production.
- B. Change the command to `dbt build --select state:modified` so only changed models build.
- C. Move the CI job into the Production environment so it can reuse the production manifest.
- D. Add `--full-refresh` so incremental models don't have to scan history.

Sample Question

Your CI job runs `dbt build` on every pull request and now takes as long as the nightly production build. The job's environment has no deferral configured.

What should you change?

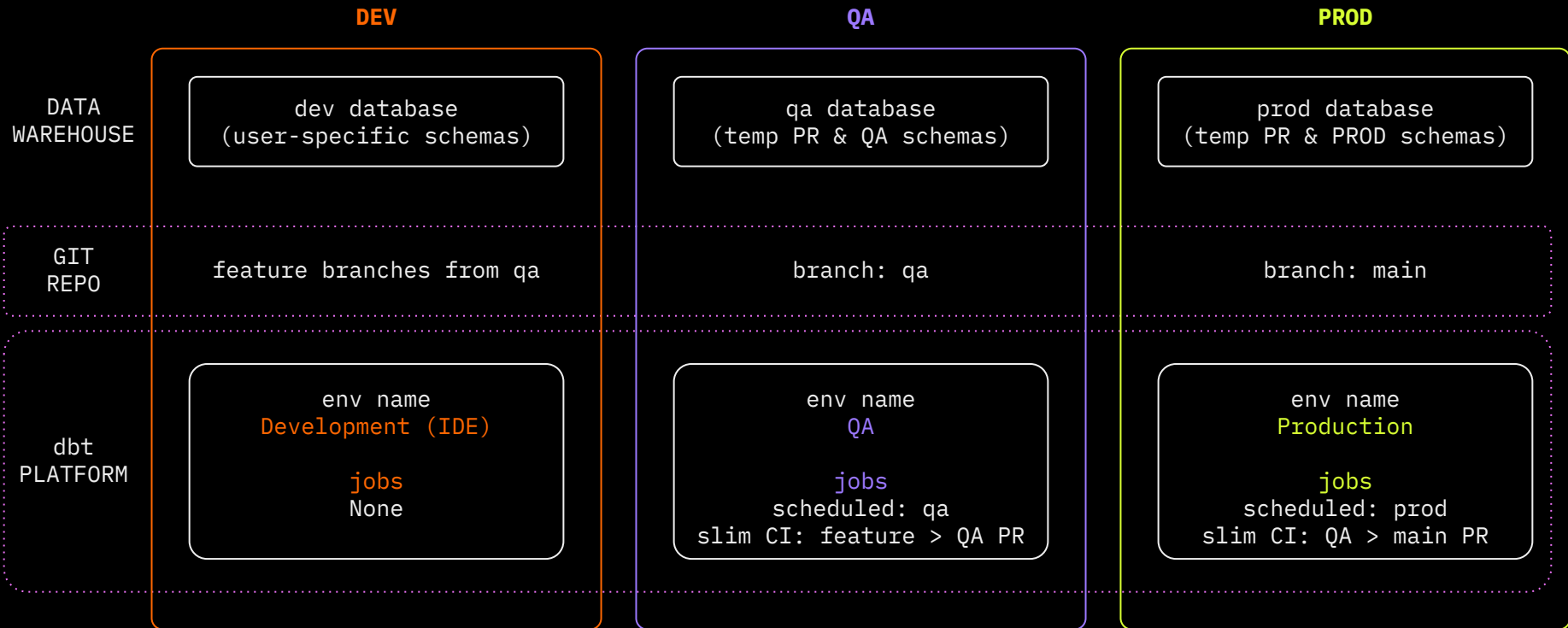
- A. Change the command to `dbt build --select state:modified+` and set the job's comparison environment to **Production**.
- B. Change the command to `dbt build --select state:modified` so only changed models build.
- C. Move the CI job into the Production environment so it can reuse the production manifest.
- D. Add `--full-refresh` so incremental models don't have to scan history.

Sample Question

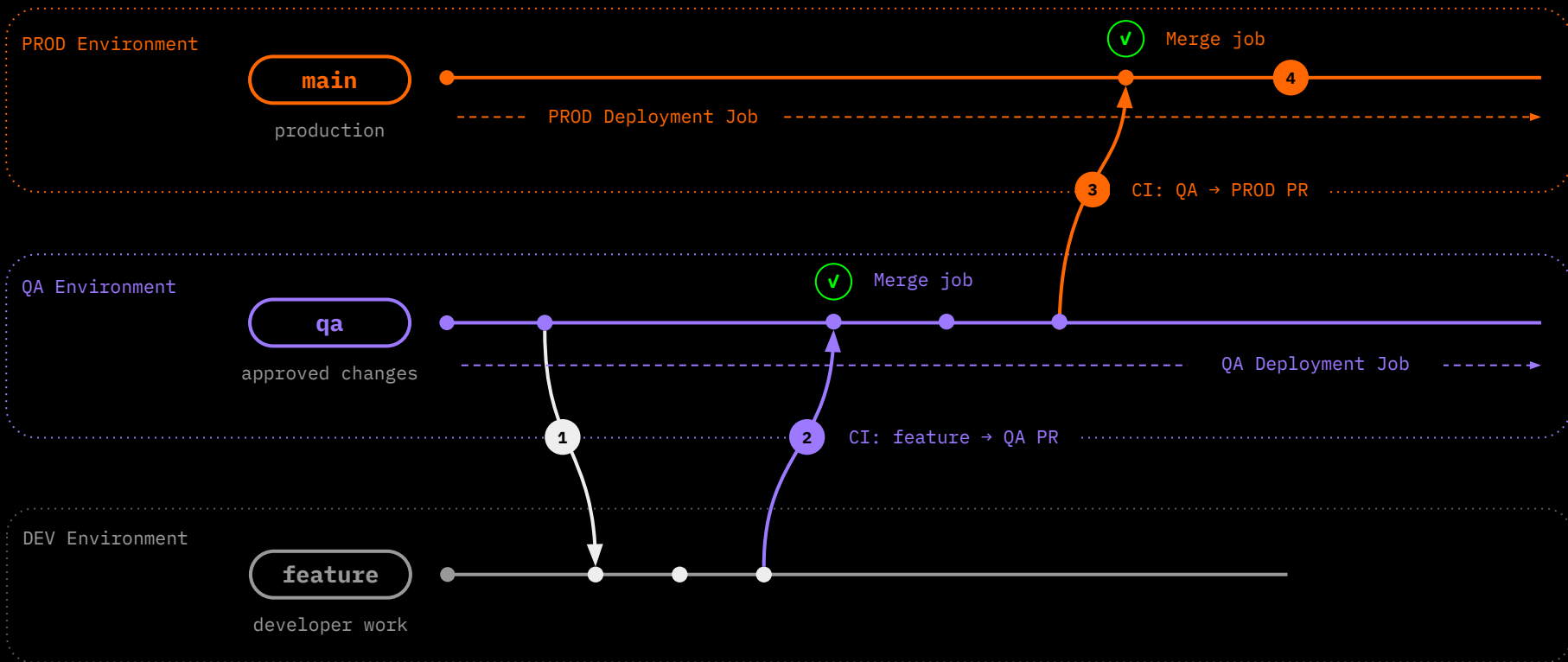
Explanation

Two things have to be true for Slim CI. The selector `state:modified+` builds changed models and their downstream dependents; dropping it (2) ships untested downstream breakage. And `state:` needs something to compare against: in the dbt platform that's the job's comparison environment, which supplies the production manifest and the defer target. Without it the selector has no prior state. Moving CI into Production (3) would build unreviewed PR code into production schemas. `--full-refresh` (4) increases runtime.

Deployment landscape



Deployment Architecture



Hands-On #3

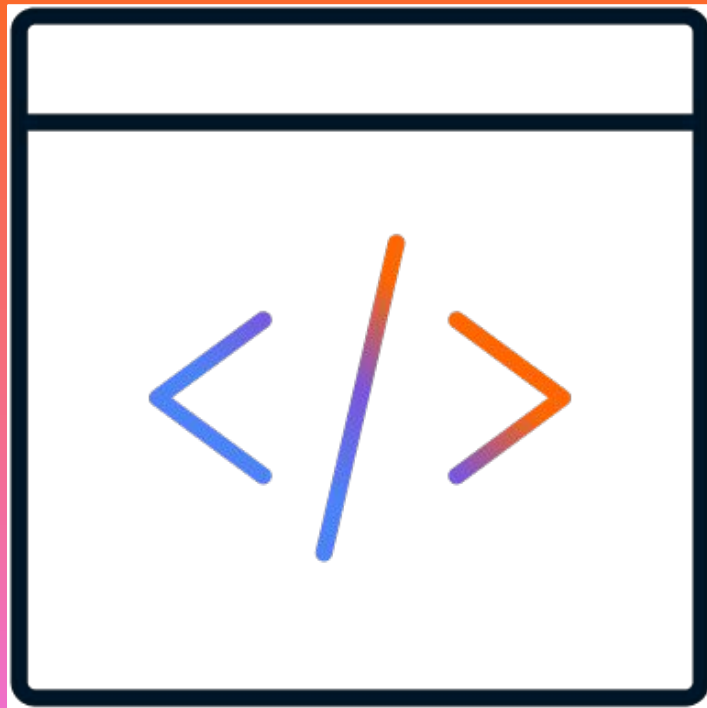
Job Design

Configure a production environment:

1. Navigate to **Orchestration**→**Environments**
2. Click into the Production Environment
3. Click **Settings** then **Edit**
4. Check "Only run on custom branch"
5. Set branch to `<your_branch_name>`, then **Save**

Create a job to run on a schedule:

1. Navigate to **Orchestration**→**Jobs**
2. Click the **Production Job** in the Production environment
3. Click **Settings** and then **Edit**
4. The command can remain as `dbt build`
5. Configure the trigger to a schedule of your choosing and **Save**
6. Go back to the job and **Run now** to manually trigger ensuring success
7. Explore the run details (Summary, Lineage, Model timing, Artifacts)





04

Security & monitoring

- Authenticating with dbt
- Access control in dbt
- Private Networking
- Webhooks
- Notifications (Slack, Teams, Webhooks)
- [DEMO] Licenses and Roles
- [HOL] Webhooks; API

Security overview – trust.getdbt.com

End-to-End Encryption

Always-encrypted communications via TLS 1.2.



Fully Encrypted Storage

Data at rest is always encrypted using AES-256. Keys are stored using AWS KMS.



Strong Authentication

SAML 2.0 integration for Single Sign-On and easy user & group management.



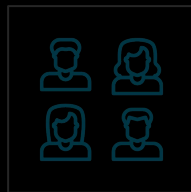
Data Storage

dbt platform is not a data warehouse. We store information about your transform jobs and users, but do not store your actual data.



Role-Based Access Control

Access and permissions to dbt platform Projects are managed through Group assignment.



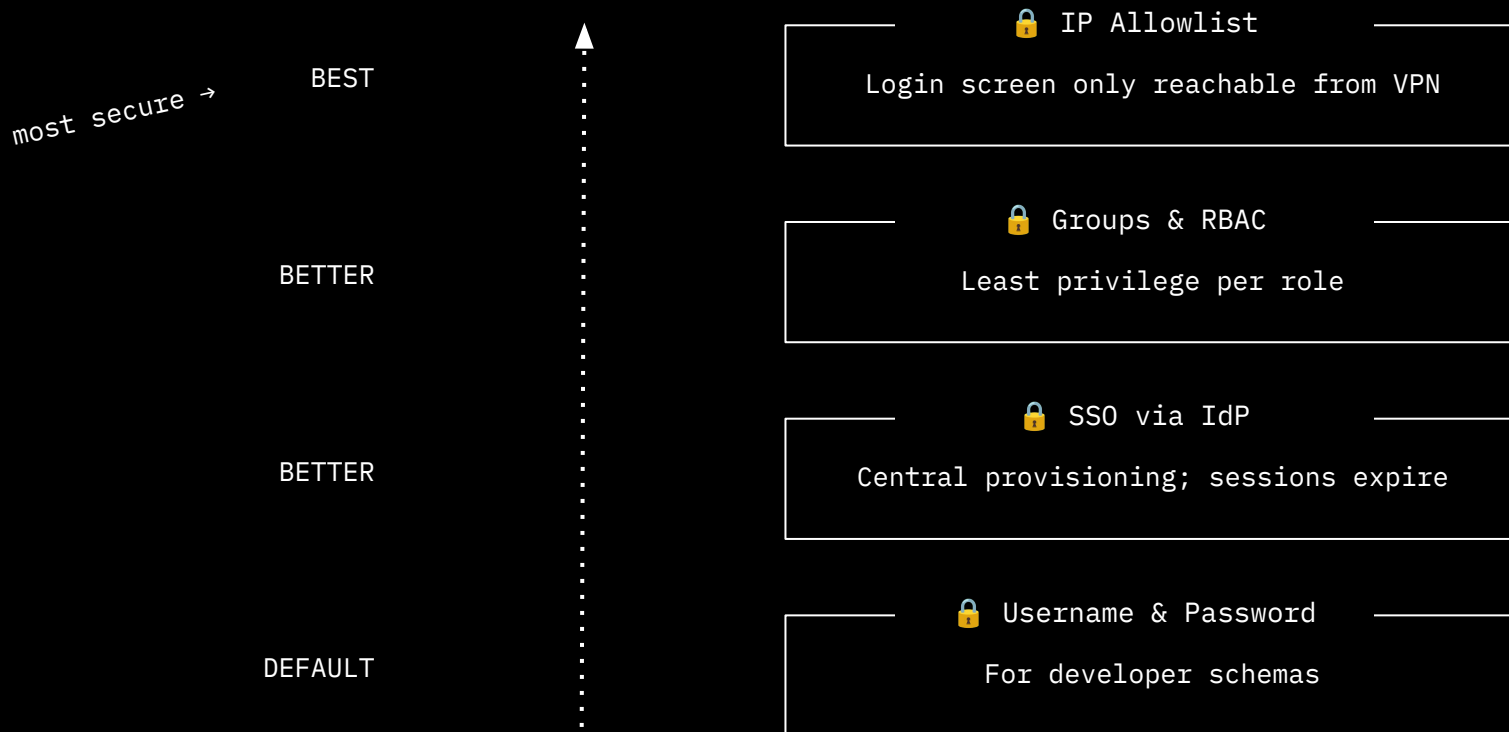
Availability & Recovery

dbt platform is hosted as a dedicated managed service on AWS with guaranteed 99.9% uptime availability



Network security options

How users should log in to the platform



Advanced compliance features

As organizations grow, dbt architectures need more than connectivity—they require strong controls over data flow. Four key features support secure, compliant operations at scale:

- **Multi-region setups:** Run dbt close to your data for compliance and performance.
- **PrivateLink integration:** Keep traffic private by routing through secure cloud networks.
- **IP allowlisting:** Limit access to approved users and networks only.
- **Data residency options:** Store metadata and processes in specific geographies for privacy and compliance.

Multi-region & data residency

Multi-region

- Run dbt close to your data and users
- Meet compliance standards (GDPR, HIPAA)
- Reduce legal risk, expand into new markets, reassure international customers

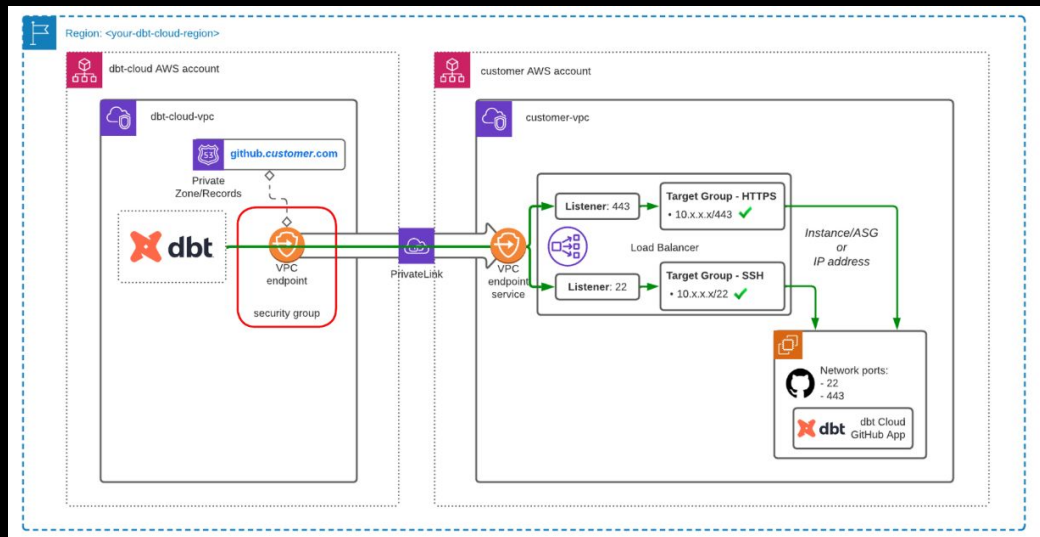
Data residency

- Pin dbt accounts to specific regions
- Metadata, logs, credentials stay there
- Prevent data from crossing borders into jurisdictions with different regulations

US · EMEA · APAC · Japan · Virtual Private

PrivateLink

- Create a private, direct link between dbt and your warehouse
- Keep all traffic off public internet
- Supports strict privacy, network isolation, and certifications (SOC 2, ISO 27001, local laws)
- Benefits: Reduced latency · Scalability · Simplified network architecture · Cost-effective

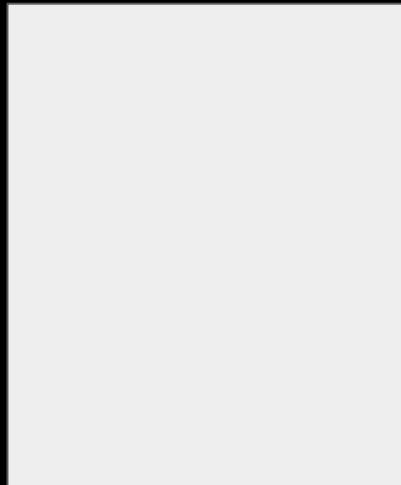


IP Allowlisting

Restrict traffic and enhance security

- Restrict access to dbt and data warehouses to known, approved IP addresses
- Set for both inbound (users → dbt) and outbound (dbt → warehouse)
- Blocks all external access except from pre-approved networks

→ Prevents unauthorized logins, reduces risk from phishing, helps to meet compliance requirements



Snowflake authentication patterns

	PERSONAL (developer)	JOB (Service account)
default	Username & password	Username & password
better	OAuth (SSO redirect)	Key pair
better	Least privilege	Least privilege + auto rotation
better	IP allowlisted	IP allowlisted
best	PrivateLink	PrivateLink

↓ increasing security ↓

Sample Question

Why should you avoid using SSO-based connections for staging and production deployment environments in dbt platform?

- A. They restrict the ability to manage environment variables required for production deployments.
- B. They cannot connect to cloud data warehouses due to incompatible authentication protocols.
- C. They rely on authentication sessions that can expire or require re-authentication, leading to unexpected job failures in automated deployments.
- D. They can only be used by users with the Account Admin role.

Sample Question

Why should you avoid using SSO-based connections for staging and production deployment environments in dbt platform?

- A. They restrict the ability to manage environment variables required for production deployments.
- B. They cannot connect to cloud data warehouses due to incompatible authentication protocols.
- C. They rely on authentication sessions that can expire or require re-authentication, leading to unexpected job failures in automated deployments.
- D. They can only be used by users with the Account Admin role.

Sample Question

Explanation

It is best practice to utilize a service account for deployment credentials. dbt platform supports authenticating via OAuth in Development environments only, where users can authorize their Development credentials using Single Sign-On. Once a user has authorized dbt platform with their data platform, via their identity provider, the user will receive (behind the scenes) an Access Token which can then be used to open a data platform connection and execute queries in the dbt platform IDE. The lifetime of the token is dictated by the data platform integration. When a user's token expires, the user will need to re-authorize with the data platform to continue development in dbt platform, therefore it is best practice to utilize a service account for deployment credentials.

Users, groups & licenses

DEFAULT GROUPS

Owner	Account admin – full privileges
Member	General, limited billing & security
Everyone	Catch-all, no permissions

→ create custom groups for real control, manage via IdP

LICENSE TYPES

Developer	Full dev, needs seat
Analyst	Dev, needs seat
IT	Security & billing admin
Read-Only	View only, no seat

→ grant the least privilege a user needs

we recommend deleting the default groups and replacing them with your own custom group setup, ideally mapped from SSO groups: `DBT_GROUP_<TEAM_NAME>_<PROJECT_NAME>_<ROLE_NAME>`

Demo #2

Licenses & Roles

1. Review the [Permission Sets](#) documentation
2. Determine 1-2 permission levels you would like to grant
3. Navigate to [Account settings](#) → [Groups & licenses](#)
4. Create a new group, name it and assign these permissions
5. Save it
6. Navigate to [Account settings](#) → [Users](#)
7. [Invite users](#) to your account by adding in their email addresses
8. Add them to this new group before sending the invitations



Monitoring & alerting

Monitor jobs & set up event-driven alerts

MONITORING

- **Source freshness** – Monitor data governance with data source freshness snapshots
- **Artifacts** – use artifacts generated by dbt builds to power health reports
- **Run visibility** – View run history to identify improvements to scheduled jobs

ALERTING

- **Job notifications** – Receive [email·Slack·Teams] notifications for jobs [success·warnings·fails·cancel]
- **Model notifications** – Receive email notifications for models & tests as they're built
- **Webhooks** – Send events from dbt jobs to **other systems**

Hands-On #4

Configure a webhook

Generate an endpoint:

1. Navigate to [beeceptor](#) and create a mock server to simulate API endpoint behaviour

Create a webhook:

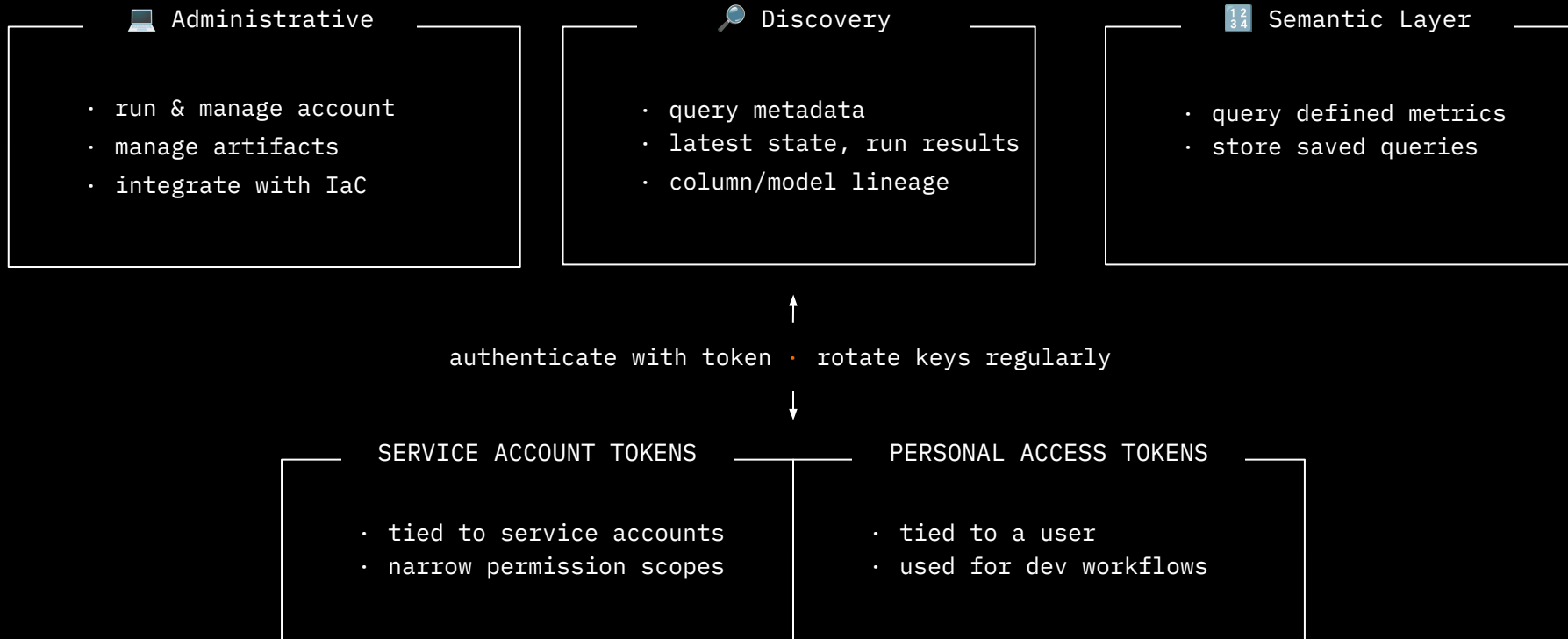
1. Navigate to **Account Settings**→**Webhooks** and **Create webhook**
2. Add relevant details and paste the beeceptor endpoint you created
3. **Save** and **Test endpoint**; navigate to [beeceptor](#) to see the notification

Test the webhook:

1. Perform the action in your webhook
2. Navigate to [beeceptor](#) to see the webhook in action



dbt Platform APIs



Hands-On #5

Call the dbt Admin API

Obtain your dbt account details:

1. Navigate to **Account Settings**→**Account** and make a note of **Account ID** and **Access URL**
2. Navigate to **API tokens**→**Service tokens** and **Create service token** giving the **Account Admin** permission set
3. **** Copy and save your service token before leaving the page ****

Call the dbt Administrative API:

1. Visit the jobs API docs [url](#) to review what's needed to trigger a job
2. Navigate to **Orchestration**→**Jobs**, pick a job and make note of the **job_id** (after /jobs/ in the page URL)
3. Enter your service token, **account_id** and **job_id** on the jobs API [url](#) and edit the **body** as needed
4. **Send API Request** and check for a **200 OK** response and your dbt platform for a triggered run



Sample Question

An external orchestrator needs to trigger a dbt job, and when the run finishes, a downstream system needs to be notified. No human credentials may be involved.

Which pairing is correct?

- A. A personal access token to trigger the job via the Administrative API, and a job notification email routed to the downstream system.
- B. A service token with a narrowly scoped permission set to trigger the job via the Administrative API, and a webhook subscribed to run completed events pointing at the downstream system's endpoint.
- C. A service token to trigger the job via the Discovery API, and a webhook subscribed to run completed events.
- D. A personal access token that polls the Discovery API on a schedule to detect the run's completion.

Sample Question

An external orchestrator needs to trigger a dbt job, and when the run finishes, a downstream system needs to be notified. No human credentials may be involved.

Which pairing is correct?

- A. A personal access token to trigger the job via the Administrative API, and a job notification email routed to the downstream system.
- B. A service token with a narrowly scoped permission set to trigger the job via the Administrative API, and a webhook subscribed to run completed events pointing at the downstream system's endpoint.
- C. A service token to trigger the job via the Discovery API, and a webhook subscribed to run completed events.
- D. A personal access token that polls the Discovery API on a schedule to detect the run's completion.

Sample Question

Explanation

Personal access tokens are tied to a user, so they break when that person's access changes – automation uses service tokens with least-privilege scopes. Triggering runs is the Administrative API; the Discovery API is read-only metadata (state, run results, lineage) and cannot trigger anything, which rules out 3 and 4. Webhooks are the event-driven path for machine-to-machine notification; notifications target people, not systems, which rules out 1.



05

Governance & Discovery

- dbt Mesh
- Model governance
- Multi-Project collaboration
- dbt Catalog
 - Performance and recommendations
 - Lineage
 - Catalog search (dbt & warehouse)
- [HOL] Model Governance; Cross-project References
- [DEMO] dbt Catalog

Repository structure

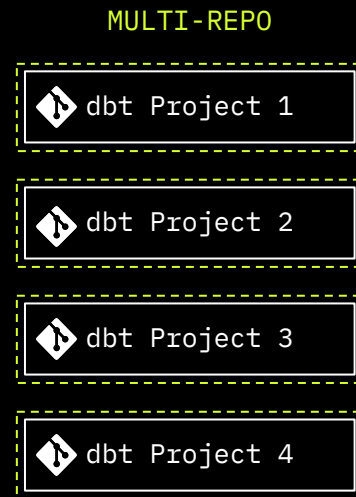
Mono-repo & multi-repo architecture

Mono-repo

- All dbt models, resources, macros, and configs managed within a *single* dbt project
- One set of models, tests, docs in one codebase

Multi-repo

- dbt resources split across multiple repos
→ *separate* dbt projects
- Each repo usually domain-oriented (eg. finance, marketing) or environment-specific (prod vs. dev)
- Teams own separate dbt projects/repos



Mono-repo vs. Multi-repo

When to use each

	MONO-REPO	MULTI-REPO
best for	Small, centrally managed team with strong business alignment	Large, domain-oriented teams with separation of duties
✓ pros	• easy search & onboarding • streamlined CI/CD • less duplication	• team & domain isolation • easier audit & permissions • simpler per-domain environments
✗ cons	• unwieldy at scale • more merge conflicts • hard to isolate changes	• duplication across repos • harder to share code • more complex CI/CD

★ neither is “right” – fit it to your org ★

Challenges with mono-projects

Monoliths

- Large complex solutions spanning multiple disparate solutions or teams

Unclear boundaries

- Complex connections, official sources are difficult to find, ownership

Brittle solutions

- Changes are difficult and the code becomes hard to maintain

Numerous projects

- Changes are difficult and the code becomes hard to maintain

Why use a
multi-project
architecture?

Multi-project architecture

...and when to use it

Multi-project collaboration enables organizations to break down analytics development into multiple dbt projects, each with its own codebase, governance, environment, and permissions.

Best when:

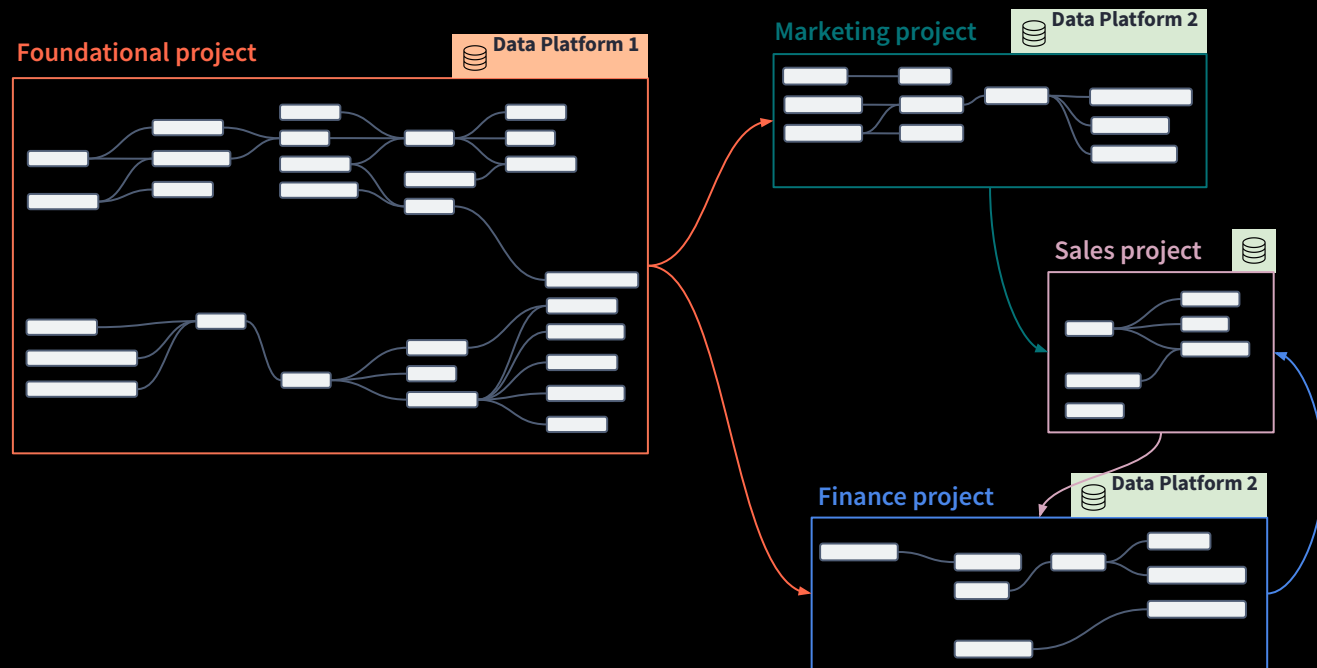
- Organization is large, domain-oriented, or operates under strict separation of duties.
- Different teams require autonomy, separate release cycles, and granular permissions.
- Modular, domain-based analytics models (e.g., data mesh principles) are needed.
- Security, scalability, and governance are priorities (e.g., regulated industries, data residency).

What is dbt Mesh?

dbt Mesh is the architecture supporting cross-project collaboration, domain ownership, and federated governance – including model versioning, contracts, and modularization.

Cross-platform dbt Mesh

dbt Mesh enables governed collaboration across multiple connected **dbt projects** and/or **data platforms**



dbt Mesh – benefits & key takeaways

SCALABILITY

- break big into modular
- share assets

GOVERNANCE

- managed dependencies
- clear owners
- enforced rules

PARALLELISM

- teams ship in parallel
- fewer bottlenecks

KEY TAKEAWAYS

- for large, domain-oriented orgs wanting autonomy + governance
- links projects for controlled sharing
- reach for it when security, scale, and regulation matter

Components of model governance

how do you safely implement cross-project dependencies at scale?

01

Model Groups

assign model collections to specific owners / groups

03

Model Contracts

guarantee the shape of your models

02

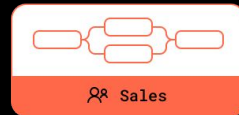
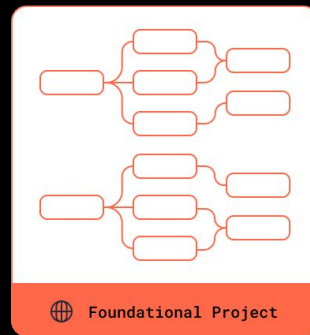
Model Access Modifiers

configure each model with a specific access policy

04

Model Versions

pin specific versions of your models



Model groups

- **Model groups** organise your models within a DAG by assigning collections to a specific owner
 - **Model groups** are designated within a yaml file nested under a **groups** key
 - Under the **groups** key, name the group and add owner information
- ★ Each model can belong *only* to one group

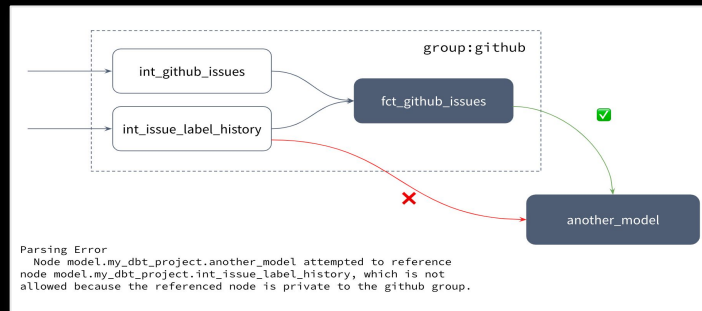
```
groups:  
  - name: Finance Group  
    owner:  
      name: Person's Name  
      email: TheirEmail@email.com  
  - name: Marketing  
    owner:  
      name: Marketing Group  
      email: marketing@jaffle.com
```

```
models:  
  - name: stg_jaffle_shop__customers  
    group: marketing  
  - name: stg_jaffle_shop__orders  
    group: revenue  
  - name: dim_customers  
    group: marketing
```

Model access policies

Models can be configured with specific access policies at the model or model group level

- **public** – can be accessed by models in any group, package, or project
- **protected (default)** – can be accessed by models in same project or group
- **private** – can only be accessed by models in same group

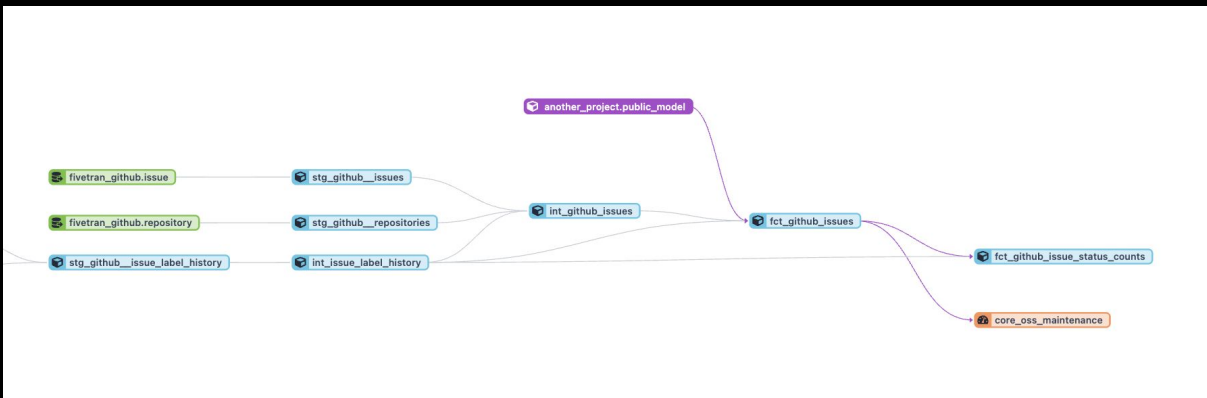


Collaborate across multiple projects

- As a model *producer* → share **public models** for use by others in the organization
- As a model *consumer* → reference models from another project
- Configure **cross-project job triggers** to ensure timely builds

```
fct_github_issues.sql  x
models > marts > github > fct_github_issues.sql

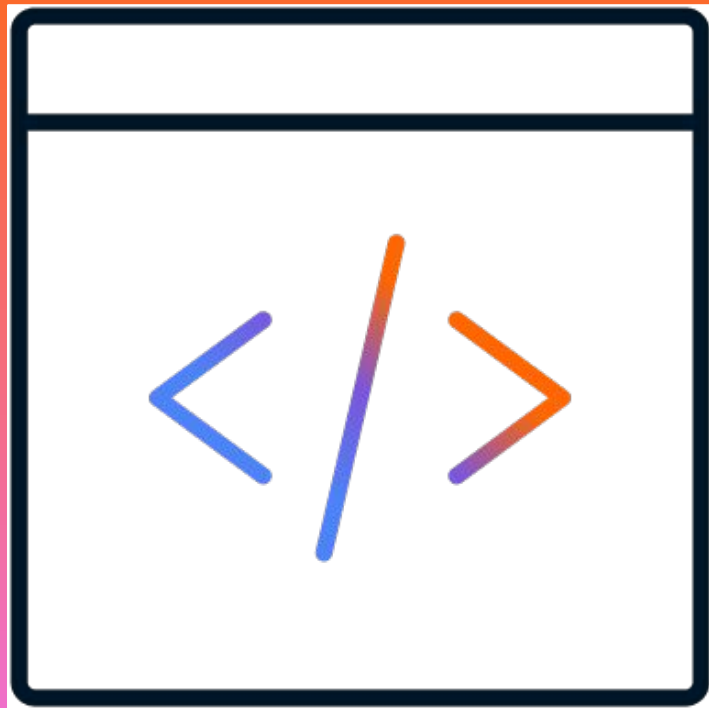
1  with public_model as (
2
3      select * from {{ ref('another_project', 'public_model') }}
4
5  ),
6
7  issues as (
8
```



Hands-On #6

Cross-project References

1. Navigate to `Foundational Project→Orchestration→Jobs` and ensure a `Staging` environment deploy job has run successfully with a `manifest.json`
2. Navigate to `Foundational Project→Studio→dbt_project.yml` and make a note of the dbt project name
3. Navigate to `Marketing Project→Studio→dependencies.yml` and add a dependency to `my_new_project` – save the file
4. Check the lineage to validate the public models have been found
5. Create a downstream model using cross-project ref



Sample Question

A 40-person data org has finance, marketing, and supply-chain teams. Each wants its own release cadence and its own permissions, but all three build on a shared set of conformed dimensions owned by a central platform team. Everything currently lives in one project in one repo, and merge conflicts are constant.

What is the recommended architecture?

- A. Keep the mono-repo and use model groups plus `private` access modifiers to isolate each team's models.
- B. Split into a foundational project owning the conformed dimensions plus one project per domain, with each domain project declaring the foundational project in `dependencies.yml` and consuming its `public` models.
- C. Split each domain into its own repo and have each domain re-implement the conformed dimensions locally, so no cross-project dependency is needed.
- D. Keep one project but create a separate deployment environment per domain so each team can deploy on its own schedule.

Sample Question

A 40-person data org has finance, marketing, and supply-chain teams. Each wants its own release cadence and its own permissions, but all three build on a shared set of conformed dimensions owned by a central platform team. Everything currently lives in one project in one repo, and merge conflicts are constant.

What is the recommended architecture?

- A. Keep the mono-repo and use model groups plus `private` access modifiers to isolate each team's models.
- B. Split into a foundational project owning the conformed dimensions plus one project per domain, with each domain project declaring the foundational project in `dependencies.yml` and consuming its `public` models.
- C. Split each domain into its own repo and have each domain re-implement the conformed dimensions locally, so no cross-project dependency is needed.
- D. Keep one project but create a separate deployment environment per domain so each team can deploy on its own schedule.

Sample Question

Explanation

Separate release cadences plus separate permissions plus domain ownership is exactly the multi-project case. Groups and access modifiers (1) organise a DAG and control reference paths, but they operate within a project – they give you neither independent release cycles nor permission boundaries. Re-implementing shared dimensions (3) is the duplication cost multi-project architecture exists to avoid. Extra deployment environments (4) don't confer separate codebases or ownership, and the merge conflicts remain.

Thought exercise

Cross-project reference scenario

- Your team uses models built *upstream* by the core engineering team.
- Your models rely on specific configurations (expected columns, data types, etc.)

→ How can you guarantee that the shape of the models built upstream is consistent for your team?

→ What happens if the upstream team makes “breaking” changes to models that your models depend on?

Model contracts

- Every model can have a **contract** that defines guarantees around data shape
- To create a model contract, add a **contract:** configuration to the model yaml
- List each expected column, along with its data type
- Add a constraints property to create an even stricter contract

```
models:  
  - name: dim_customers  
    access: public  
    config:  
      contract:  
        enforced: true  
    columns:  
      - name: customer_id  
        data_type: number  
        constraints:  
          - type: not_null  
    ...
```

Model versioning

- Work on model changes without disrupting *consumer* workflows
- Downstream consumers can work ahead of changes – or – pin specific versions that work for their project
- Simply add a **latest_version** property to your models yaml
- **defined_in** tracks where a file moves to
- **alias**: provides a name to identify a model with
- **include**: keeps fields from previous versions
- **old column configs**: indicates the original config of the changed column

```
models:
  - name: dim_customers
    latest_version: 2
    columns:
      - name: customer_id
        data_type: number
      - name: number_of_orders
        data_type: number
    versions:
      - v: 2
      - v: 1
      defined_in: dim_customers_v1
      config:
        alias: dim_customers
      columns:
        - include: *
          exclude: number_of_orders
        - name: number_of_orders
          data_type: text
```

Thought exercise

Model versioning scenario

<https://docs.getdbt.com/reference/resource-properties/versions?version=2.0&name=v2>

Which columns for each model version?

models/customers.yml

```
models:
  - name: customers
    columns:
      - name: customer_id
        description: Unique identifier for this table
        data_type: text
        constraints:
          - type: not_null
        data_tests:
          - unique
      - name: customer_country
        data_type: text
        description: "Country where the customer currently lives"
      - name: first_purchase_date
        data_type: date

versions:
  - v: 4

  - v: 3
    columns:
      - include: "*"
      - name: customer_country
        data_type: text
        description: "Country where the customer first lived at time of first purchase"

  - v: 2
    columns:
      - include: "*"
      exclude:
        - customer_country

  - v: 1
    columns:
      - include: []
      - name: id
        data_type: int
```

Ref-ing & running a model version

→ How do you ensure downstream models are using the right model version?

ref a version

```
{{ ref('dim_customers', v=2) }}
```

run a version

```
dbt run -s dim_customers.v2
```

default behavior

latest version will be used

Hands-On #7

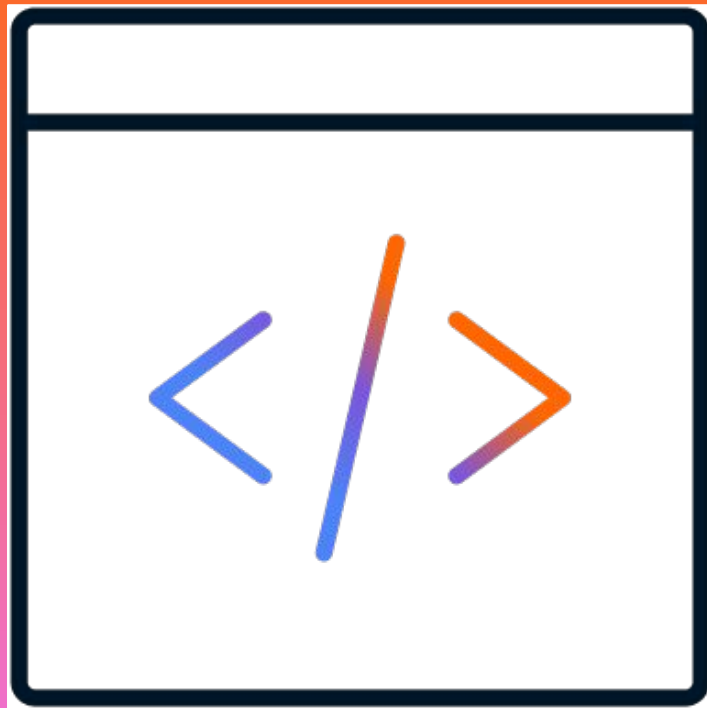
Model contracts & versioning

Enforcing a model contract:

1. Navigate to `Foundational Project-Studio-models-governance-marts-hands_on` and enforce a contract on the `hands_on_orders` model
2. Change the data type of the `order_id` column to `int` and build the model. What happened?
3. Rename the `tax_paid` column to `tax_due` and build the model. What happened?

Model versioning

1. Create a model called `hands_on_orders_v2.sql` with the `tax_paid` column aliased as `tax_due`
2. Add `latest_version` and `versions` configs
3. Create a new file in that folder called `downstream_model.sql` which selects `*` from `hands_on_orders`. Save and observe lineage. Which version is ref'd?



Hands-On #7

Model contracts & versioning

Model versioning (continued):

4. Change the `latest_version` to 2 in `hands_on_orders.yml`. What happens?
5. Rename the `hands_on_orders.sql` model to `hands_on_orders_v1.sql`. Save and observe lineage. Which version is ref'd now?

Aliasing versioned models:

1. Compile `downstream_model.sql` and observe the built table name. Why change this? How?
2. Add `alias` config to `hands_on_orders.yml`
3. Re-compile and observe change



Sample Question

You own a public model `dim_customers` with an enforced contract. You need to change `number_of_orders` from `number` to `text`. Three downstream projects consume the model and you cannot coordinate a simultaneous release with all three.

What is the correct sequence?

- A. Change the data type in place; the enforced contract will fail the build and alert consumers before anything breaks downstream.
- B. Create `dim_customers_v2.sql` with the new type, add a `versions` block while leaving `latest_version: 1`, let consumers move to `ref('dim_customers', v=2)` on their own schedule, then promote `latest_version` to 2.
- C. Set `access: protected` until all three projects have migrated, then set it back to `public`.
- D. Add a new `number_of_orders_text` column alongside the old one and deprecate the old column in a later release.

Sample Question

You own a public model `dim_customers` with an enforced contract. You need to change `number_of_orders` from `number` to `text`. Three downstream projects consume the model and you cannot coordinate a simultaneous release with all three.

What is the correct sequence?

- A. Change the data type in place; the enforced contract will fail the build and alert consumers before anything breaks downstream.
- B. Create `dim_customers_v2.sql` with the new type, add a `versions` block while leaving `latest_version: 1`, let consumers move to `ref('dim_customers', v=2)` on their own schedule, then promote `latest_version` to 2.
- C. Set `access: protected` until all three projects have migrated, then set it back to `public`.
- D. Add a new `number_of_orders_text` column alongside the old one and deprecate the old column in a later release.

Sample Question

Explanation

Contracts guarantee the shape of a model; versions are the mechanism for changing that shape without a coordinated release. Publishing v2 while `latest_version` stays at 1 means unpinned refs keep resolving to v1, so consumers migrate when they're ready; promoting `latest_version` later flips the default. Changing the type in place (1) is the breaking change, not a warning about it. Flipping to `protected` (3) severs the cross-project refs immediately. Shadow columns (4) work but leave permanent schema debt and bypass the governed pattern.

dbt Catalog

the dbt platform knowledge base & interactive lineage graph



Discover & visualize

- birds-eye view
- find resources & deps
- reuse, don't rebuild

Scale & context

- live, performant
- project recommendations
- health signals

Global lineage

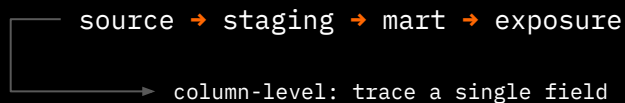
- models across projects
- lineage lenses
- global search

Search, lineage & column-level

SEARCH

- keyword + filters
- columns within models
- warehouse metadata
- across all projects
- stg / prod env filters

LINEAGE



USE CASES

- impact: "who feels this change"
- root cause: trace to the source
- spans projects in a Mesh

Model performance & recommendations

Catalog reads your metadata → a prioritized to-do list

PERFORMANCE ANALYSIS

"What's slow or breaking?"

- trends: exec time, run count, tests
- pinpoint long-running / failing models
- target cost · reliability · time

PROJECT RECOMMENDATIONS

"What's thin or undocumented?"

- test & doc coverage at a glance
- project health score
- guidance on *how* & *where* to fix
- powered by platform metadata + best practices

→ helps to produce your *low-hanging fruit* list -
where to spend your effort to cut cost & raise reliability

Demo #3

Discovery with dbt Catalog

1. Run a successful staging job
2. Open the **Foundational Project** and navigate to your staging environment and set the target branch to **catalog_hands_on**
3. Run the job, let it fail and navigate to the Catalog
4. Select the **Staging** environment and open the lineage for **Foundational Project**. Change the lens to **latest status** to observe failing models
5. Return to the Catalog page and scroll down to latest activity → issues
6. Click on a failed model and go to the **performance** tab
7. Select **Staging Job** and open the failed job to uncover the cause



Sample Question

Finance reports that a dashboard number is wrong. You need to establish which upstream column feeds it, and whether anything in that chain failed in the most recent run. The chain spans two projects in a dbt Mesh.

What gets you there fastest?

- A. Query the Administrative API for the latest run results in both projects and cross-reference model names.
- B. Open Run History in each project, filter by failure status, and inspect each run's Lineage tab.
- C. Use column-level lineage in dbt Catalog to trace the field back to its source, then apply the latest status lens on the lineage graph to spot failures in that chain.
- D. Check project recommendations for both projects to find undocumented or untested models in the chain.

Sample Question

Finance reports that a dashboard number is wrong. You need to establish which upstream column feeds it, and whether anything in that chain failed in the most recent run. The chain spans two projects in a dbt Mesh.

What gets you there fastest?

- A. Query the Administrative API for the latest run results in both projects and cross-reference model names.
- B. Open Run History in each project, filter by failure status, and inspect each run's Lineage tab.
- C. Use column-level lineage in dbt Catalog to trace the field back to its source, then apply the latest status lens on the lineage graph to spot failures in that chain.
- D. Check project recommendations for both projects to find undocumented or untested models in the chain.

Sample Question

Explanation

Column-level lineage answers "which upstream column," lineage lenses answer "what's failing," and Catalog spans projects in a Mesh – which per-project Run History (3) does not. The Administrative API (1) manages runs and account resources; lineage metadata comes from the Discovery API. Project recommendations (4) surface coverage and health gaps, not run outcomes.



06

Core exam concepts

- Exam structure
- Common Pitfalls
- Conclusion/Next steps

Exam logistics

- 65 questions · 2 hours (~1.8 min/question) · online proctored via Talview
- Passing score 65%; you see your score immediately
- \$200, but free at dbt Summit! Available in English, French and Japanese
- Valid 2 years from award date
- All questions weighted equally, 1 point each, no penalty for wrong answers. Never leave a blank
- An undisclosed number of unscored research questions are mixed in, indistinguishable from scored ones. A question that feels bizarre may simply not count; don't let it rattle you.
- Retakes allowed, but each requires a new registration fee. Reschedule or cancel free up to 24 hours before.
- Recommended background: SQL proficiency + ~6 months administering an Enterprise dbt account

Question formats

Format	What it means	What it punishes
Multiple choice	As you'd expect	N/A
Fill-in-the-blank	Type exact config, command, value, etc.	Recognising syntax but not recalling it
Matching	Column A → Column B	Partial knowledge
Hotspot	Click the right region	Knowing <i>what</i> to do but not <i>where</i> it lives
Build list	Put steps in order	Not knowing sequences
DOMC	Options shown one at a time , answer YES/NO to each. You cannot see them side by side or go back.	Elimination strategy. It simply doesn't work here

Topic coverage today

1. **Configuring dbt data warehouse connections:** §02 adapters + Hands-On #1; auth patterns (§04); the account-hierarchy reference slides
2. **Configuring dbt git connections:** §02 git providers; §03 branching strategies + custom branch config
3. **Creating & maintaining dbt environments:** §03 environments, env var precedence, Demo #1, Hands-On #2, Hands-On #3
4. **Creating & maintaining job definitions:** §03 job types, commands, runs, Hands-On #3
5. **Configuring dbt security & licenses:** §04 auth, RBAC, networking, Demo #2, Hands-On #5
6. **Monitoring & alerting for jobs:** §04 monitoring, notifications, Hands-On #4
7. **dbt Mesh & cross-project references:** §05 Mesh, model governance, Hands-On #6, Hands-On #7
8. **Configuring & using dbt Catalog:** §05 Catalog, lineage, recommendations, Demo #3

Common pitfalls

Top tips:

1. **Use the study guide thoroughly.** Nothing in the guide is out of scope
2. **Get hands-on with features you haven't used.** If you've only used a slice of the platform, go build the rest.
3. **Take the full two hours.** Subtle wording differences decide questions.

Three common traps:

1. **"Best practice" vs "possible."** Several distractors are technically achievable but not advised.
2. **Right setting, wrong level.** Many distractors put a real setting at the wrong altitude: account vs project vs environment vs job.
3. **Service token vs personal access token.** This distinction recurs across topics.

Next steps

Study guide: [dbt Architect Certification Exam Study Guide](#)

Certification page: [dbt Architect certification exam](#)

Learning path: [dbt Certified Architect Path](#)

Registration: [dbt Analytics Engineering Certification Exam](#)

Platform-specific admin courses (Snowflake · BigQuery · Databricks · Redshift),
the RBAC and SSO video series, dbt Mesh course, dbt Catalog course

[dbt Community Slack](#) for ongoing questions

Session repo and materials: <https://tinyurl.com/s26-architect>

Session survey: <https://tinyurl.com/dbt-summit-survey>

Thank you!



Please complete our
survey!

[https://tinyurl.com/
dbt-summit-survey](https://tinyurl.com/dbt-summit-survey)

 **SUMMIT**